EXPLORATORY DATA ANALYSIS

TITANIC DATASET

Exploratory Data Analysis (EDA) is the process of examining datasets to summarize key characteristics, uncover patterns, and detect anomalies. It uses statistical methods and visualizations like histograms, scatter plots, and box plots. EDA helps understand data distributions, relationships, and quality, guiding further analysis, feature engineering, and model building effectively.

Import Required Libraries

In [103…

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

IMPORT THE DATASET

In [104…

```
titanic_data=pd.read_csv('train.csv')
titanic_data
```

Out[104...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	7
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	5
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	
...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	1
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	3
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	3
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	

891 rows × 12 columns

BASIC FUNCTIONS

In [105...

```
#HEAD GIVE THE FIRST 5 ROWS FROM THE DATASET
titanic_data.head()
```

Out[105…

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0

In [106…

```
#ISNULL GIVES THE NULL AS TRUE AND ISNULL().SUM() GIVES THE COUNT OF THE NULL VA
titanic_data.isnull().sum()
```

Out[106…

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

In [107…

```
#FILLING THE NULL VALUES(NAN) WITH THE MEAN AND THE MODE FROM THOSE COLUMNS
titanic_data['Age'].fillna(titanic_data["Age"].mean(), inplace=True)
titanic_data['Cabin'].fillna(titanic_data["Cabin"].mode()[0], inplace=True)
titanic_data['Embarked'].fillna(titanic_data["Embarked"].mode()[0], inplace=True
```

In [108…

```
titanic_data
```

Out[108…

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Tick
0	1	0	3	Braund, Mr. Owen Harris	male	22.000000	1	0	A 211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000	1	0	PC 175
2	3	1	3	Heikkinen, Miss. Laina	female	26.000000	0	0	STON/O 31012
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000	1	0	1138
4	5	0	3	Allen, Mr. William Henry	male	35.000000	0	0	3734
...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.000000	0	0	2115
887	888	1	1	Graham, Miss. Margaret Edith	female	19.000000	0	0	1120
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118	1	2	W./ 66
889	890	1	1	Behr, Mr. Karl Howell	male	26.000000	0	0	1113
890	891	0	3	Dooley, Mr. Patrick	male	32.000000	0	0	3703

891 rows × 12 columns

In [109…

```
titanic_data.isnull().sum()
```

Out[109…

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

In [110…

```
#INFO()-GIVES THE SUMMARY OF NON NULL COUNT AND THE DATATYPE OF THE COLUMN
titanic_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        891 non-null    object 
 11  Embarked     891 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [111…

```
#Describe gives all the statistical information from the dataset to all the nume
titanic_data.describe()
```

Out[111…

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204
std	257.353842	0.486592	0.836071	13.002015	1.102743	0.806057	49.693
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910
50%	446.000000	0.000000	3.000000	29.699118	0.000000	0.000000	14.454
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329

In [112…

```
# value counts gives the count of apperance
columns=titanic_data.columns
```

```
for i in columns:
    value_count=titanic_data[i].value_counts()
    print(value_count)
    print('~~~~~~~~~~~')
```

```
PassengerId
1      1
599    1
588    1
589    1
590    1
      ..
301    1
302    1
303    1
304    1
891    1
Name: count, Length: 891, dtype: int64
~~~~~~~~~~~~
Survived
0    549
1    342
Name: count, dtype: int64
~~~~~~~~~~~~
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
~~~~~~~~~~~~
Name
Braund, Mr. Owen Harris                     1
Boulos, Mr. Hanna                           1
Frolicher-Stehli, Mr. Maxmillian            1
Gilinski, Mr. Eliezer                       1
Murdlin, Mr. Joseph                         1
                                          ..
Kelly, Miss. Anna Katherine "Annie Kate"    1
McCoy, Mr. Bernard                          1
Johnson, Mr. William Cahoone Jr             1
Keane, Miss. Nora A                         1
Dooley, Mr. Patrick                         1
Name: count, Length: 891, dtype: int64
~~~~~~~~~~~~
Sex
male      577
female    314
Name: count, dtype: int64
~~~~~~~~~~~~
Age
29.699118    177
24.000000     30
22.000000     27
18.000000     26
28.000000     25
             ...
36.500000      1
55.500000      1
0.920000       1
23.500000      1
74.000000      1
Name: count, Length: 89, dtype: int64
~~~~~~~~~~~~
SibSp
0    608
```

```
1    209
2     28
4     18
3     16
8      7
5      5
Name: count, dtype: int64
~~~~~~~~~~~~
Parch
0    678
1    118
2     80
5      5
3      5
4      4
6      1
Name: count, dtype: int64
~~~~~~~~~~~~
Ticket
347082      7
CA. 2343    7
1601        7
3101295     6
CA 2144     6
           ..
9234        1
19988       1
2693        1
PC 17612    1
370376      1
Name: count, Length: 681, dtype: int64
~~~~~~~~~~~~
Fare
8.0500     43
13.0000    42
7.8958     38
7.7500     34
26.0000    31
           ..
35.0000     1
28.5000     1
6.2375      1
14.0000     1
10.5167     1
Name: count, Length: 248, dtype: int64
~~~~~~~~~~~~
Cabin
B96 B98        691
G6               4
C23 C25 C27      4
C22 C26          3
F33              3
              ...
E34              1
C7               1
C54              1
E36              1
C148             1
Name: count, Length: 147, dtype: int64
~~~~~~~~~~~~
```

```
Embarked
S    646
C    168
Q     77
Name: count, dtype: int64
~~~~~~~~~~~
```

In [113…
```
# CATEGORIZING THE DATA AS NUMERICAL AND CATEGORICAL
data_types=dict(titanic_data.dtypes)
categorical_columns=[key for key,value in data_types.items() if value=='object']
numerical_columns=[key for key,value in data_types.items() if value!='object']
print(categorical_columns)
print(numerical_columns)
```

```
['Name', 'Sex', 'Ticket', 'Cabin', 'Embarked']
['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
```

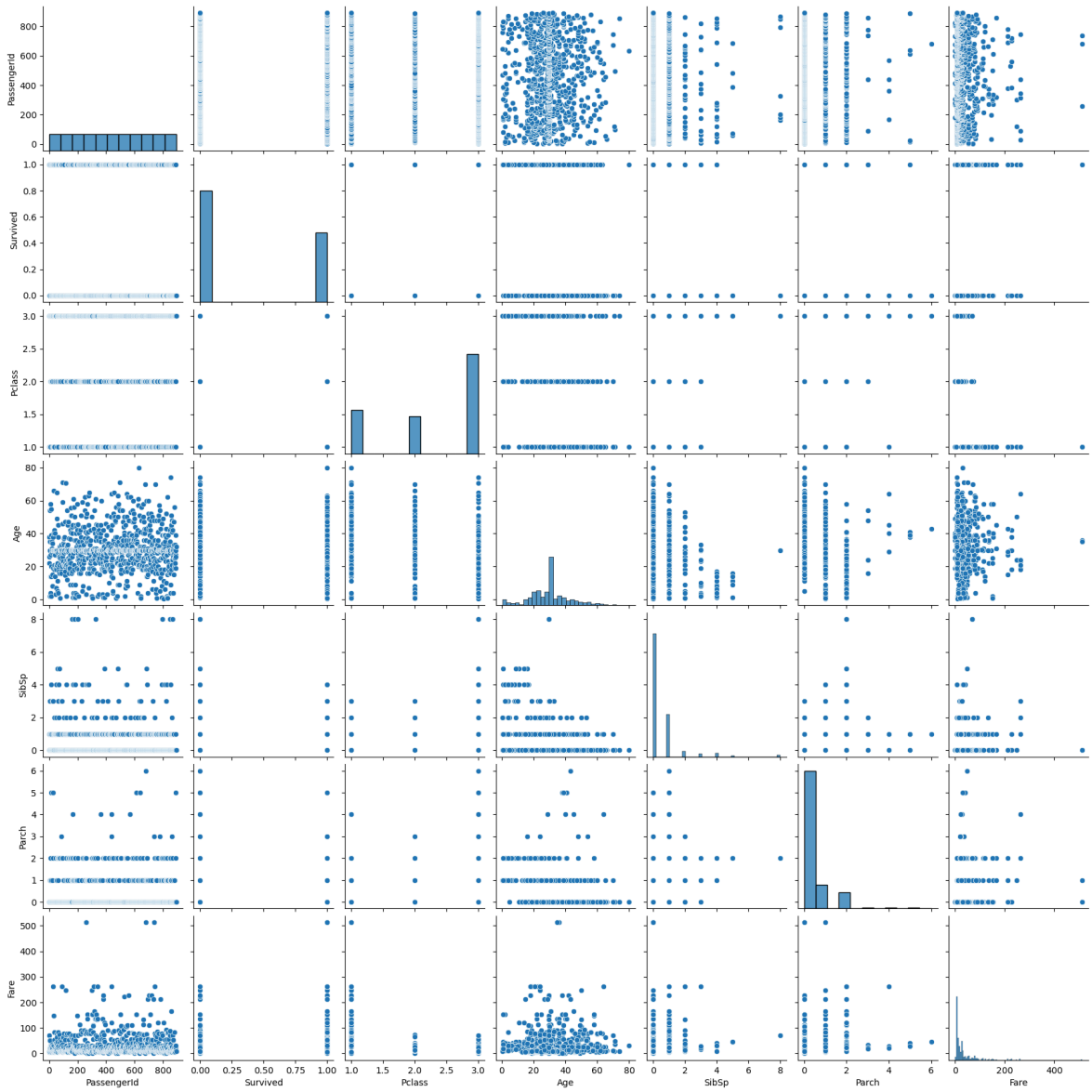
Data VISUALIZATION

PAIRPLOT

A **pairplot** is a Seaborn visualization that displays **scatterplots for every pair of numeric variables** in a dataset and histograms or KDE plots on the diagonal. It helps identify **relationships, correlations, patterns, and outliers**, making it a powerful tool in Exploratory Data Analysis for understanding variable interactions visually.

In [114…
```
#PAIRPLOT-Can perform only on numerical columns
sns.pairplot(titanic_data[['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp',
```

Out[114…
```
<seaborn.axisgrid.PairGrid at 0x18973596350>
```

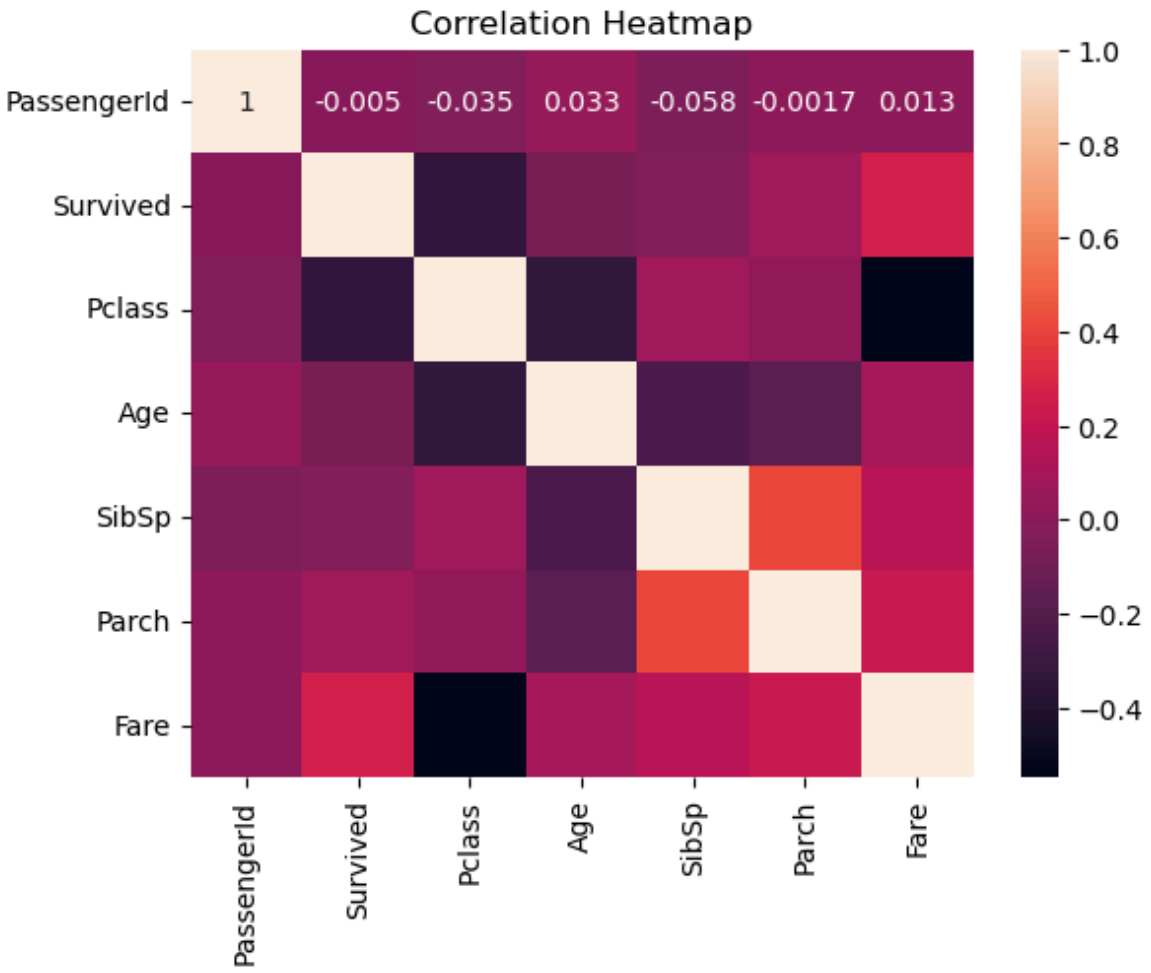


HEATMAP

A **heatmap** is a graphical representation of data using colors to show values in a matrix. It is commonly used for **correlation matrices** to visualize relationships between variables. Heatmaps help quickly identify **strong, weak, or negative correlations**, patterns, and anomalies, making them valuable in Exploratory Data Analysis.

In [115…

```
#HEATMAP
correlation=titanic_data.corr(numeric_only=True)
sns.heatmap(correlation,annot=True)
plt.title('Correlation Heatmap')
plt.show()
```



TRENDS AND RELATIONSHIPS

In EDA, **trends** show patterns or directions in data over time or across groups, while **relationships** reveal how variables influence each other. Using visualizations like scatter plots, boxplots, heatmaps, and pairplots, along with statistics like correlation and group summaries, helps uncover patterns, dependencies, and insights for better analysis.

RELATION BETWEEN NUMERICAL AND CATEGORICAL

In [116…
```
titanic_data.groupby("Pclass")["Age"].mean()
```

Out[116…
```
Pclass
1    37.048118
2    29.866958
3    26.403259
Name: Age, dtype: float64
```

RELATION BETWEEN CATEGORICAL AND CATEGORICAL

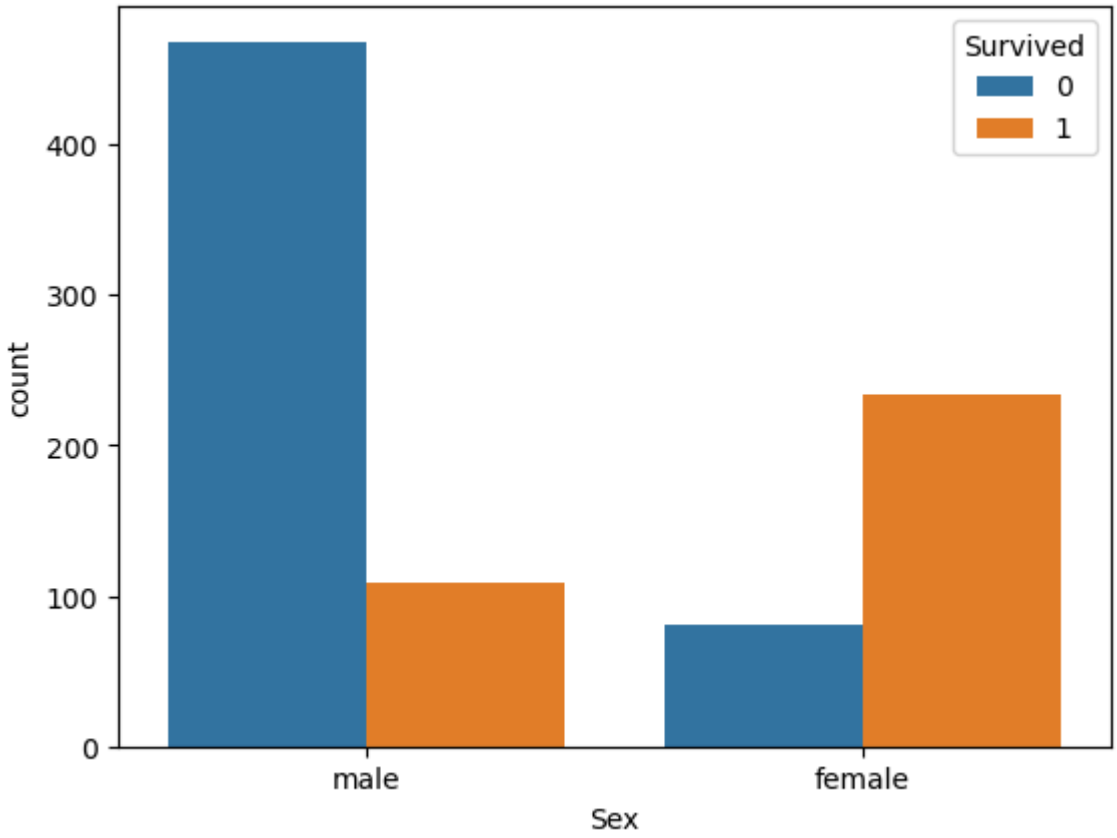
In [117…
```
pd.crosstab(titanic_data["Sex"], titanic_data["Survived"])
```

Out[117…

Survived	0	1
Sex		
female	81	233
male	468	109

In [118…

```
titanic_data['Survived'] = titanic_data['Survived'].astype(str)
sns.countplot(x="Sex", hue="Survived", data=titanic_data)
plt.show()
```



HISTOGRAM

A **histogram** is a graphical representation of data distribution. It divides numerical data into **bins** (intervals) and shows the **frequency** of values in each bin using bars. Histograms help identify patterns, trends, skewness, and outliers, making them essential for understanding the spread and shape of a dataset.

-

In [119…

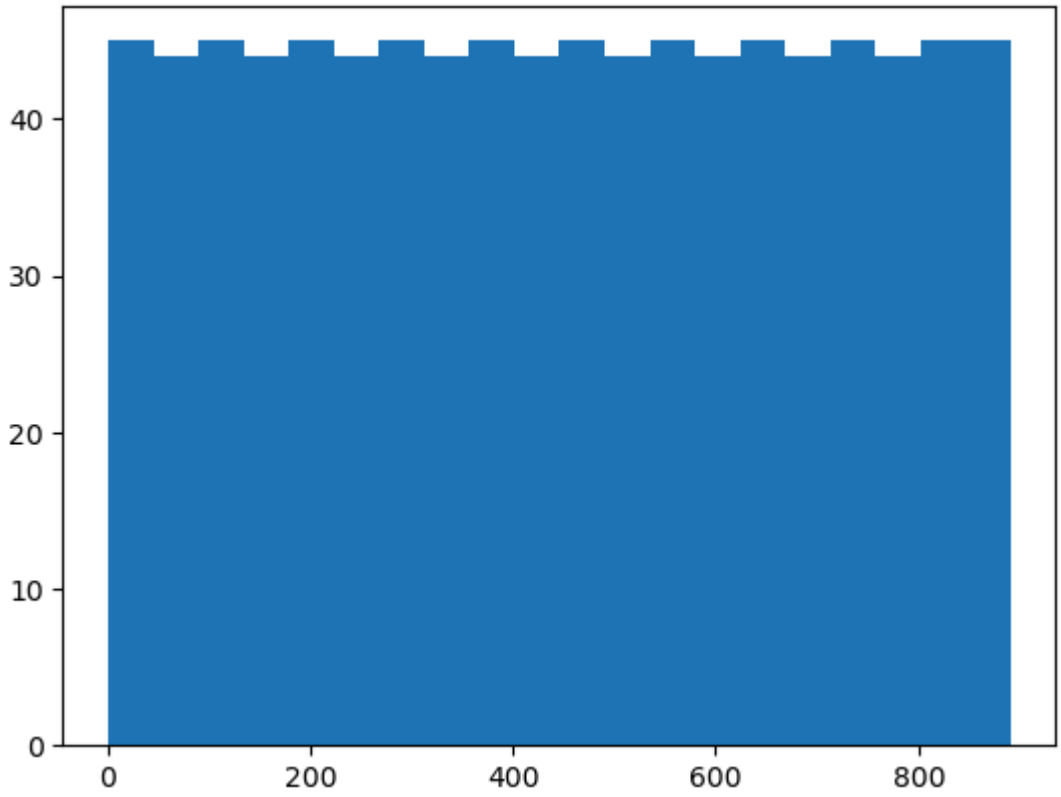
```
numerical_columns
```

Out[119…

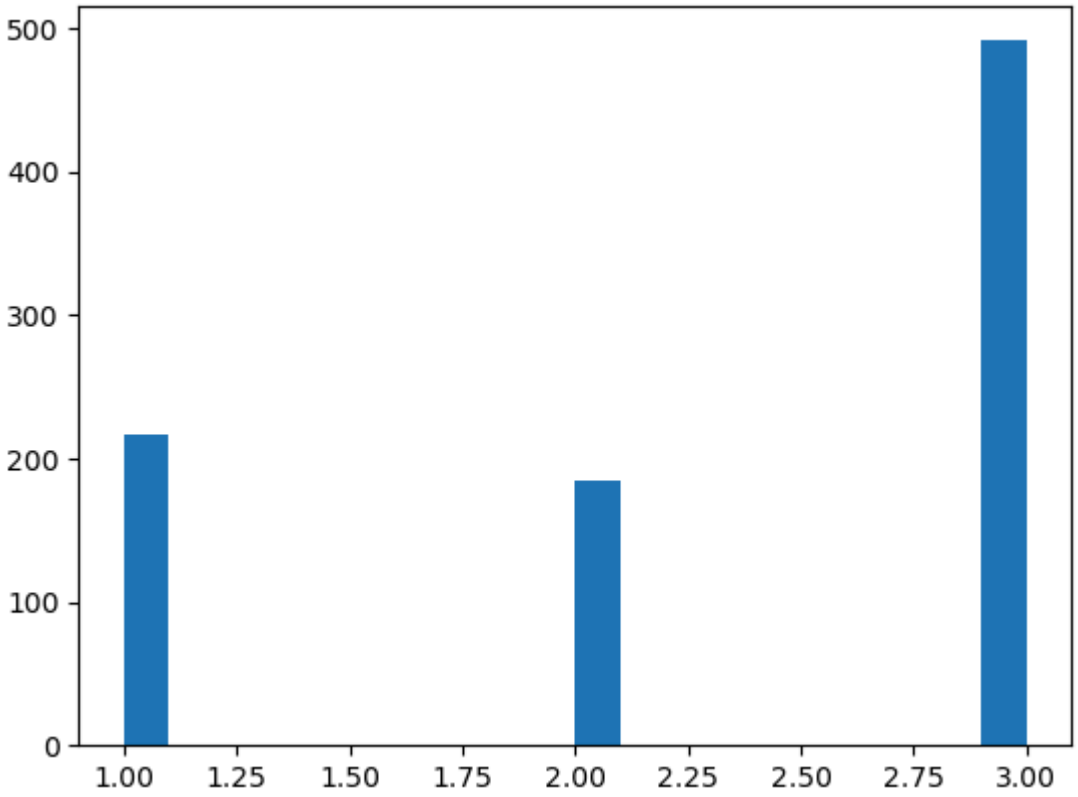
```
['PassengerId', 'Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
```

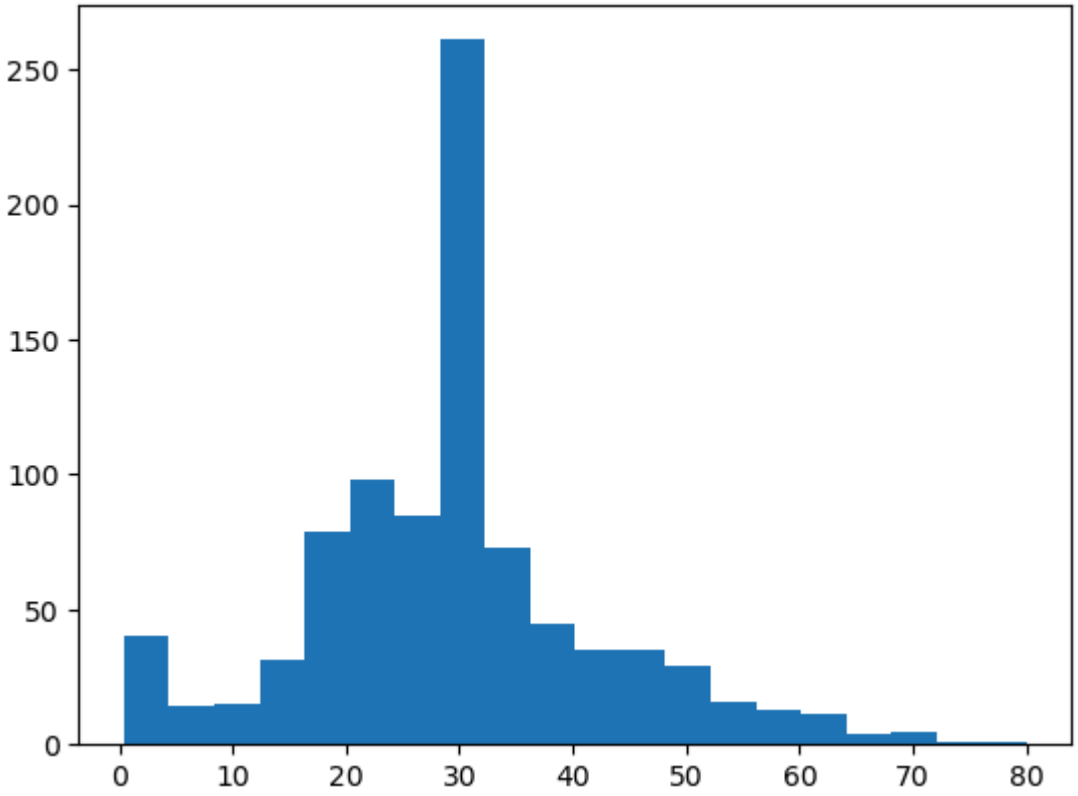
In [120…

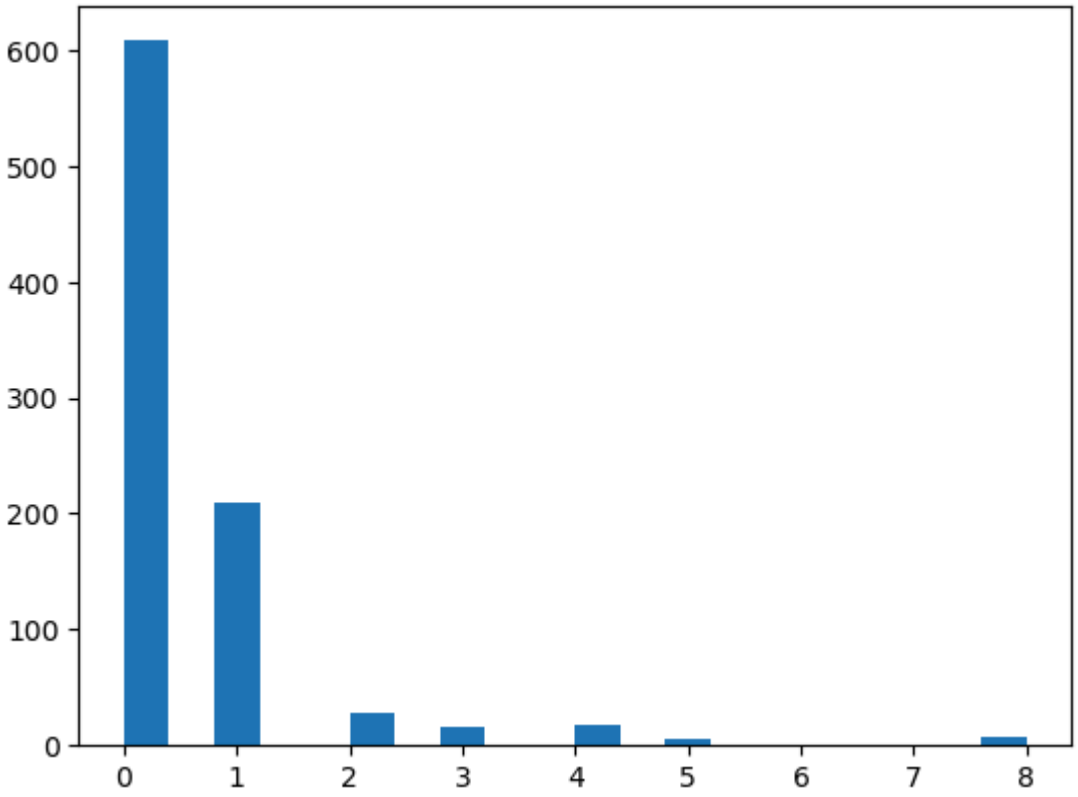
```
#HISTOGRAM FOR ALL NUMERIC COLUMNS
for i in numerical_columns:
    histogram_data=titanic_data[i]
    plt.hist(histogram_data,bins=20)
    plt.show()
```

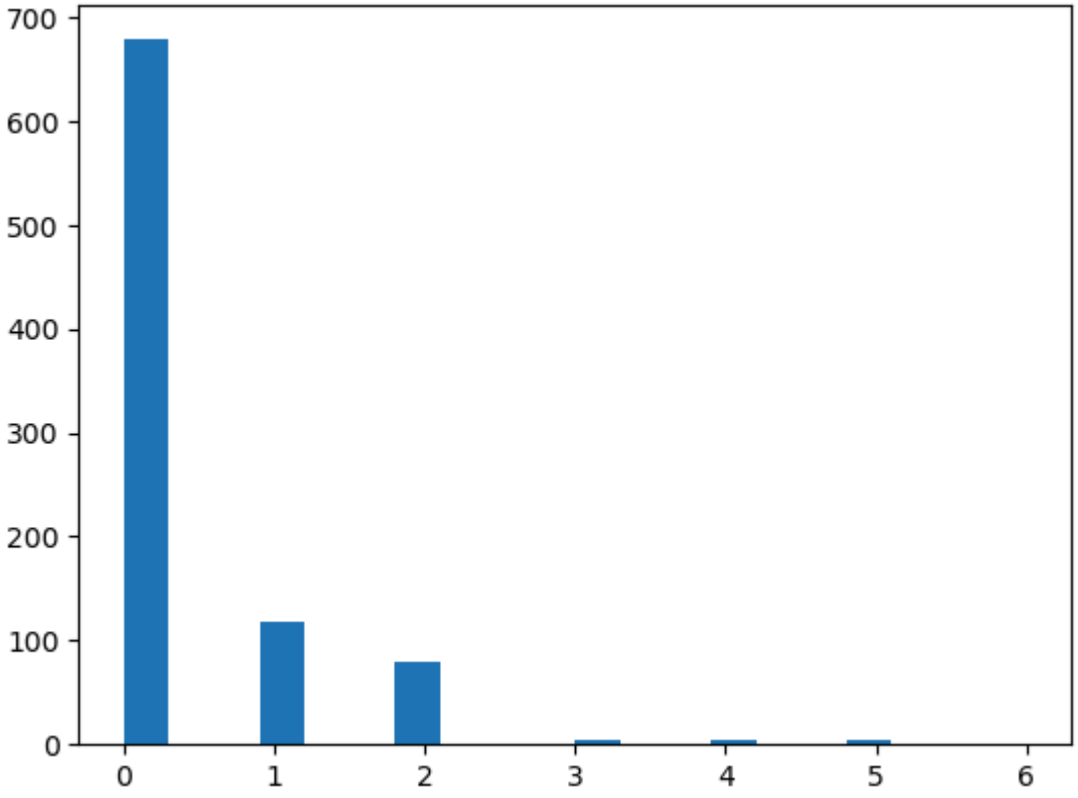

40
30
20
10
0
0
200
400
600
800

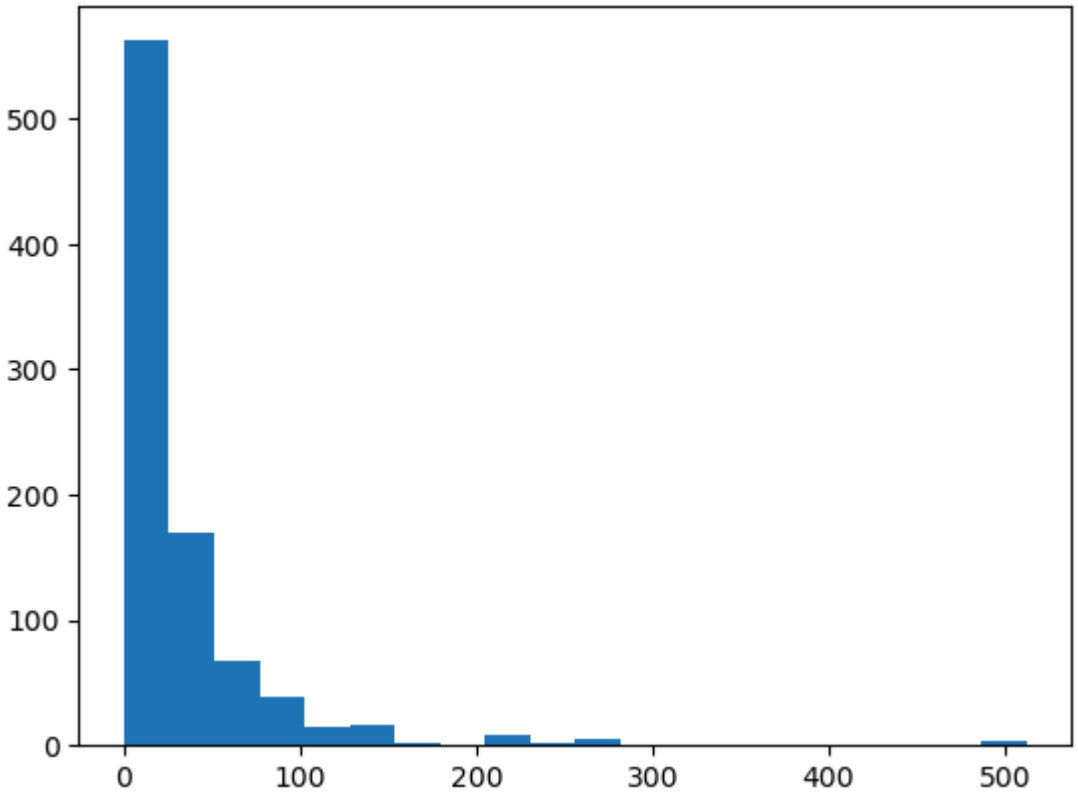

500
400
300
200
100
0
0
1


500
400
300
200
100
0
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00


250
200
150
100
50
0
0
10
20
30
40
50
60
70
80


600
500
400
300
200
100
0
0
1
2
3
4
5
6
7
8


700
600
500
400
300
200
100
0
0
1
2
3
4
5
6

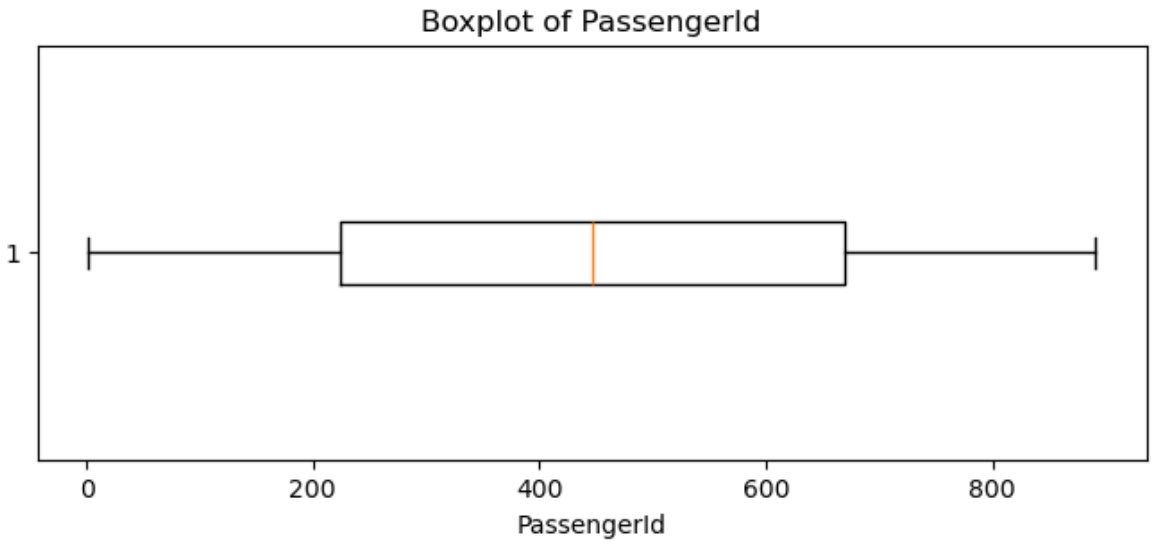


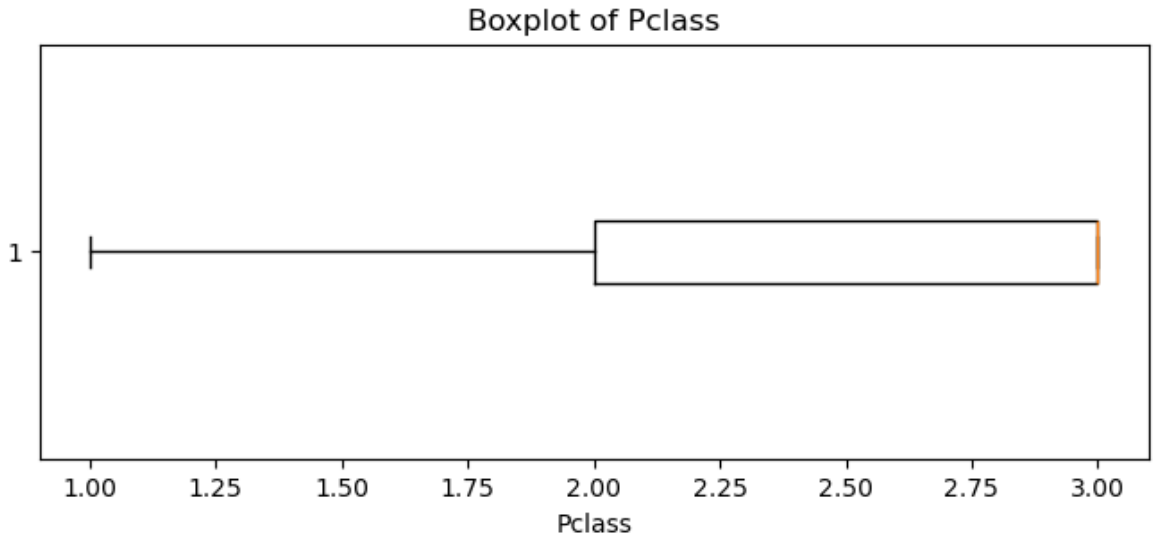
BOX PLOT

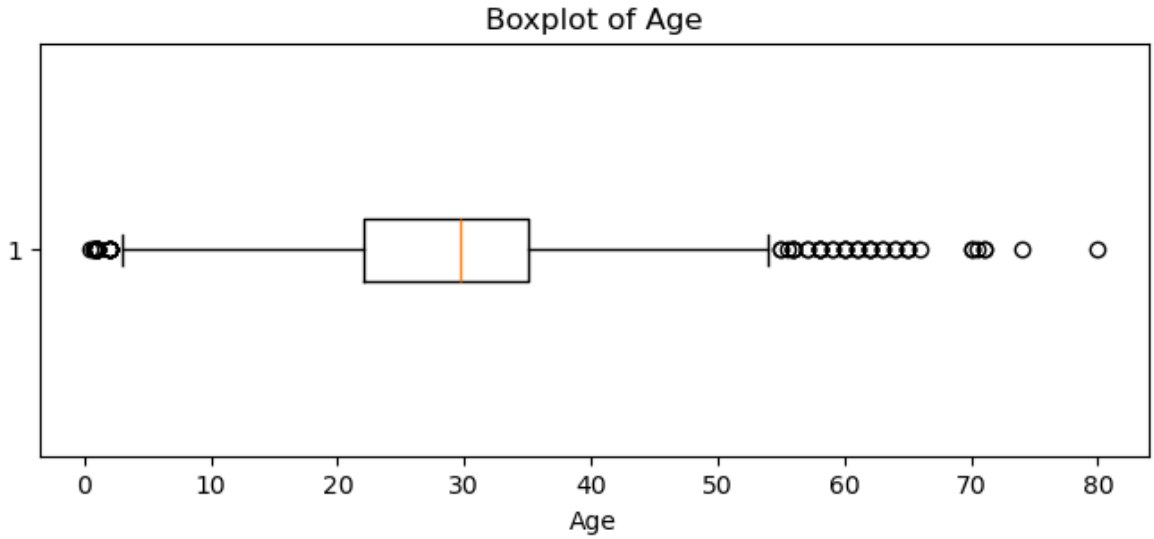
A **boxplot** is a visual summary of a dataset's distribution. It displays the **median, quartiles, minimum, maximum**, and **outliers** using a box and whiskers. Boxplots help compare groups, identify spread, detect skewness, and highlight unusual values, making them useful for exploring relationships and spotting trends in data.

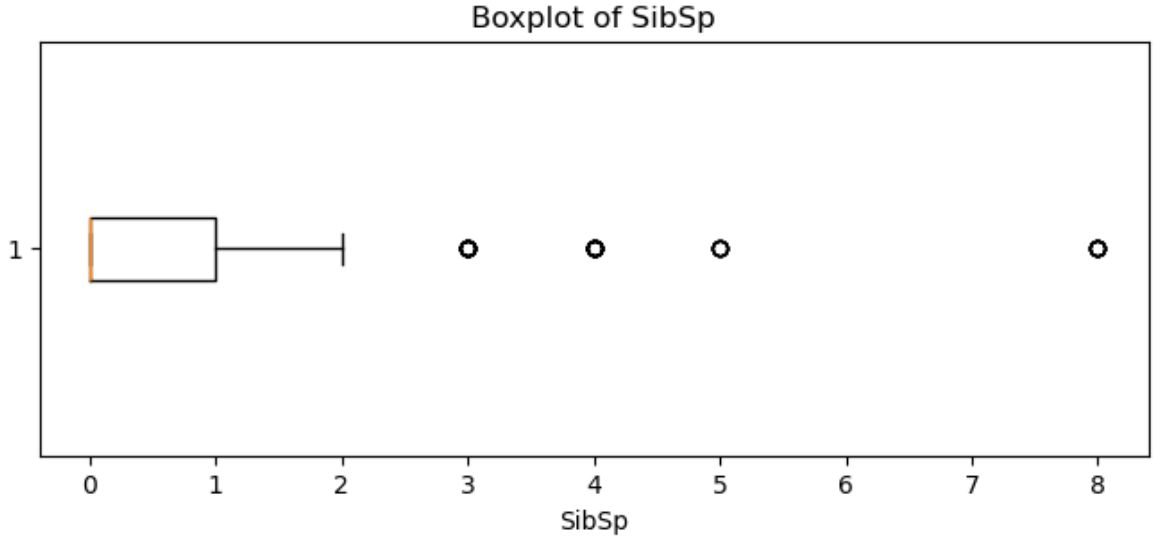
In [121…

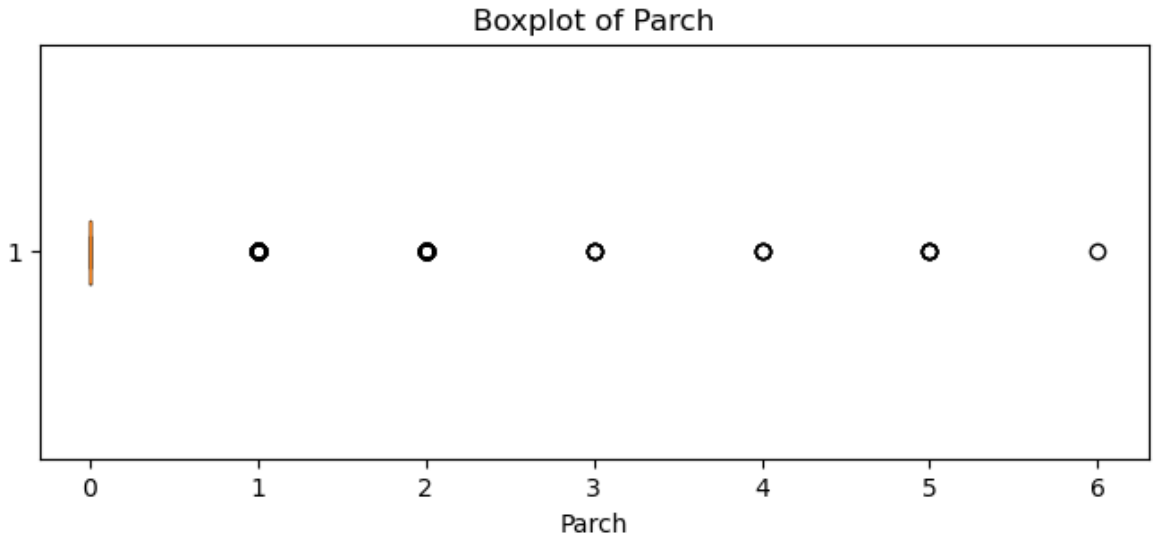
```
for col in numerical_columns:
    if pd.api.types.is_numeric_dtype(titanic_data[col]):
        plt.figure(figsize=(8, 3))
        plt.boxplot(titanic_data[col].dropna(), vert=False)
        plt.title(f'Boxplot of {col}')
        plt.xlabel(col)
        plt.show()
```

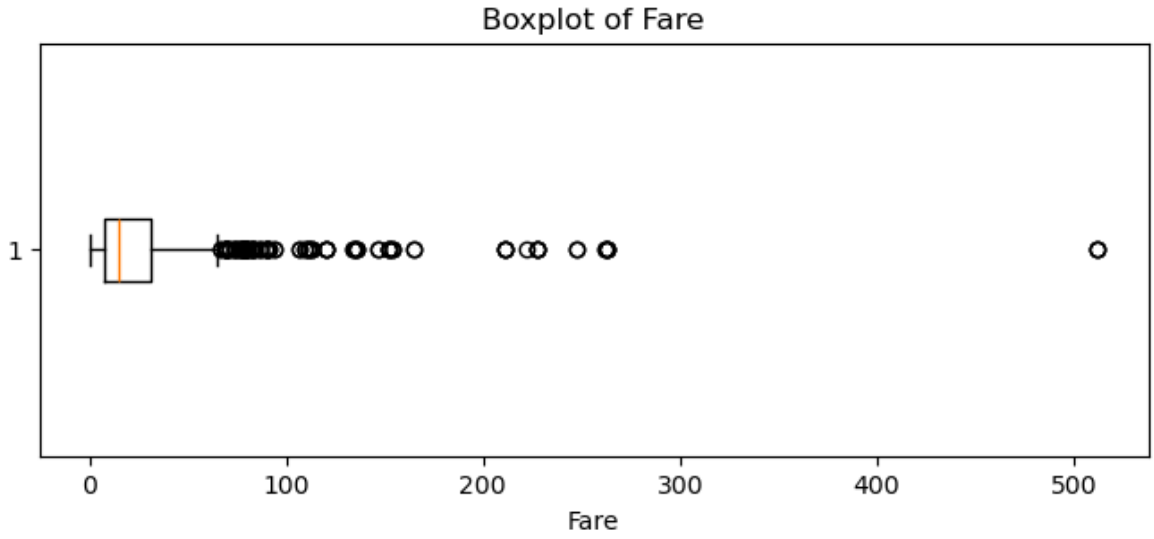



Boxplot of Pclass
1
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
Pclass


Boxplot of Age
1
0
10
20
30
40
50
60
70
80
Age


Boxplot of SibSp
1
0
1
2
3
4
5
6
7
8
SibSp



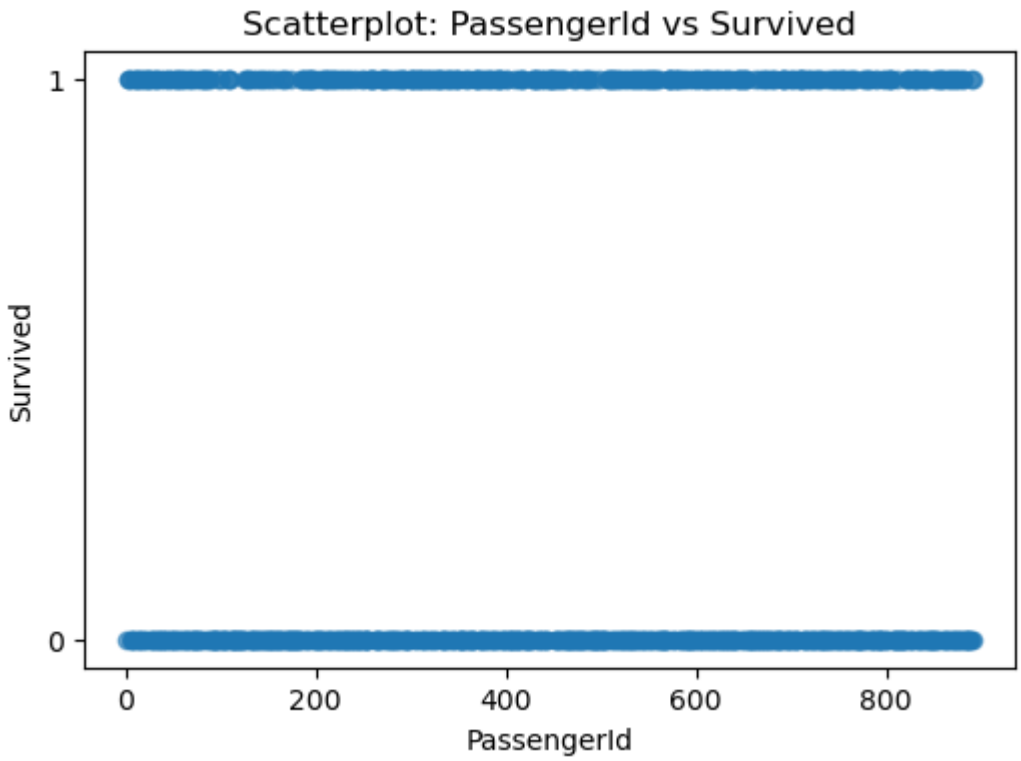


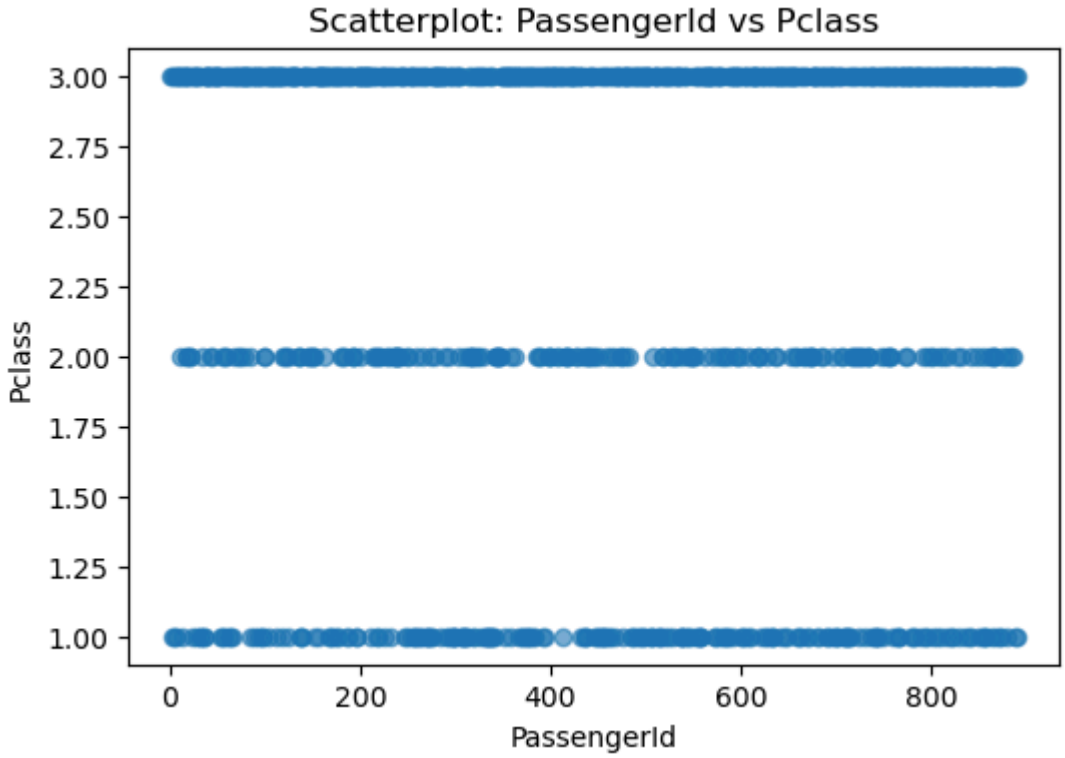
SCATTER PLOT

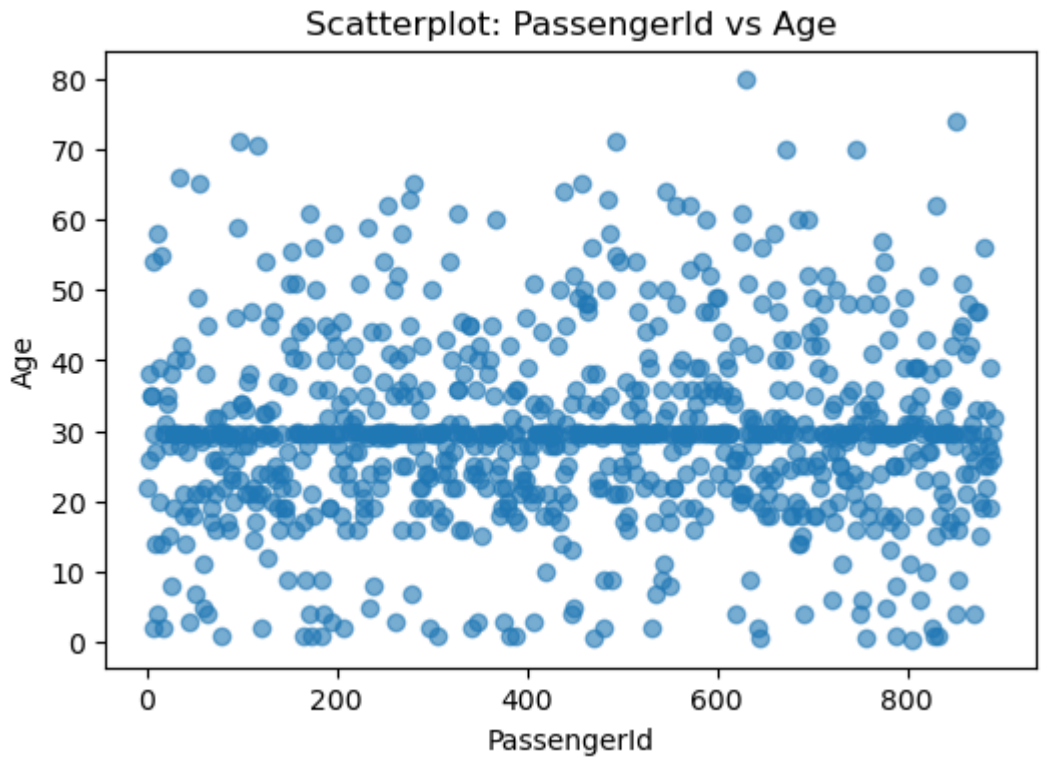
A **scatterplot** is a graph that displays the relationship between two numeric variables using points on a Cartesian plane. It helps identify **trends, correlations, clusters, and outliers**. Scatterplots are essential in EDA for visualizing how one variable changes with another, revealing patterns and potential predictive relationships.

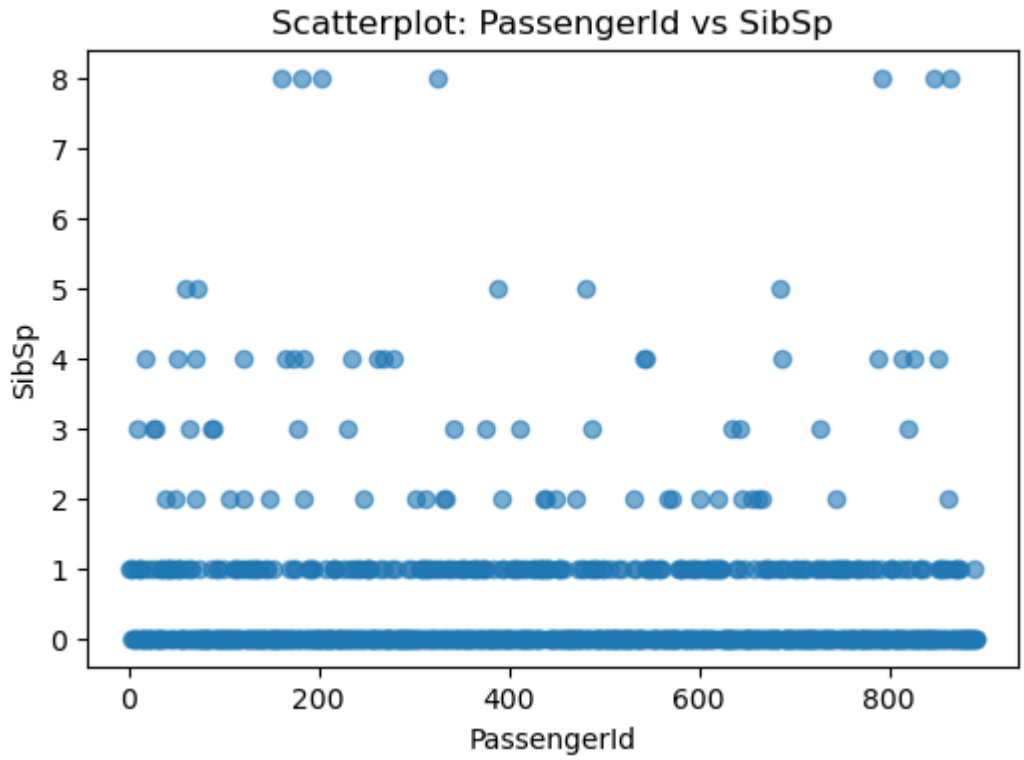
In [122…

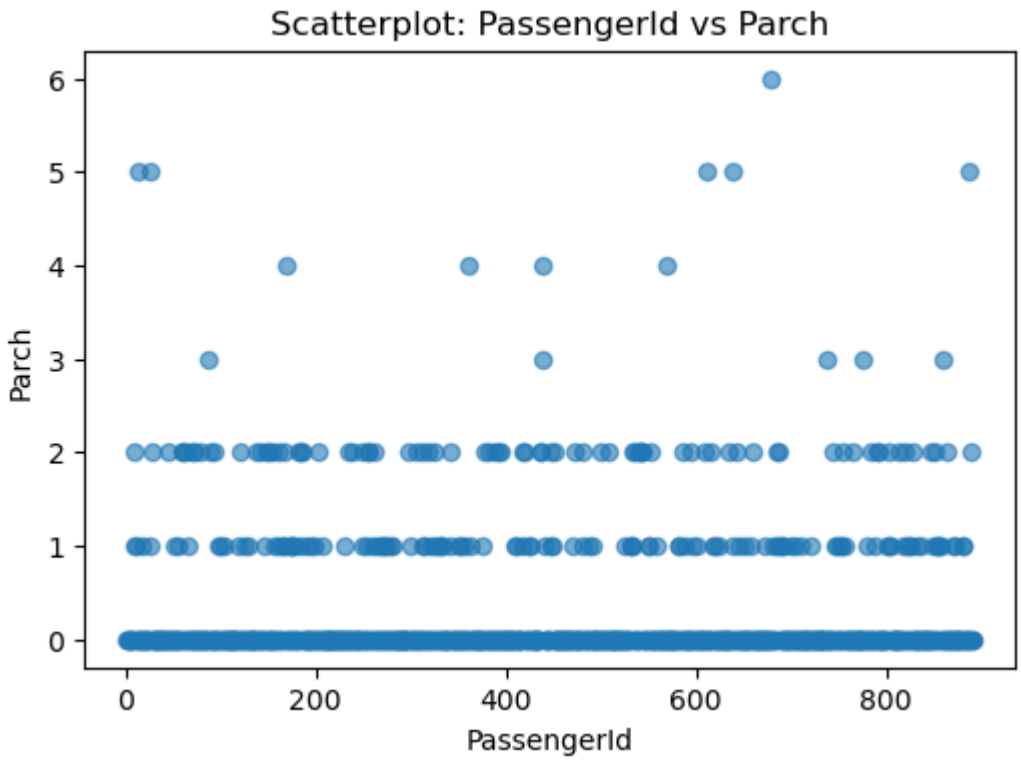
```
for i in range(len(numerical_columns)):
    for j in range(i+1, len(numerical_columns)):
        x_col = numerical_columns[i]
        y_col = numerical_columns[j]
        plt.figure(figsize=(6,4))
        plt.scatter(titanic_data[x_col], titanic_data[y_col], alpha=0.6)
        plt.xlabel(x_col)
        plt.ylabel(y_col)
        plt.title(f'Scatterplot: {x_col} vs {y_col}')
        plt.show()
```

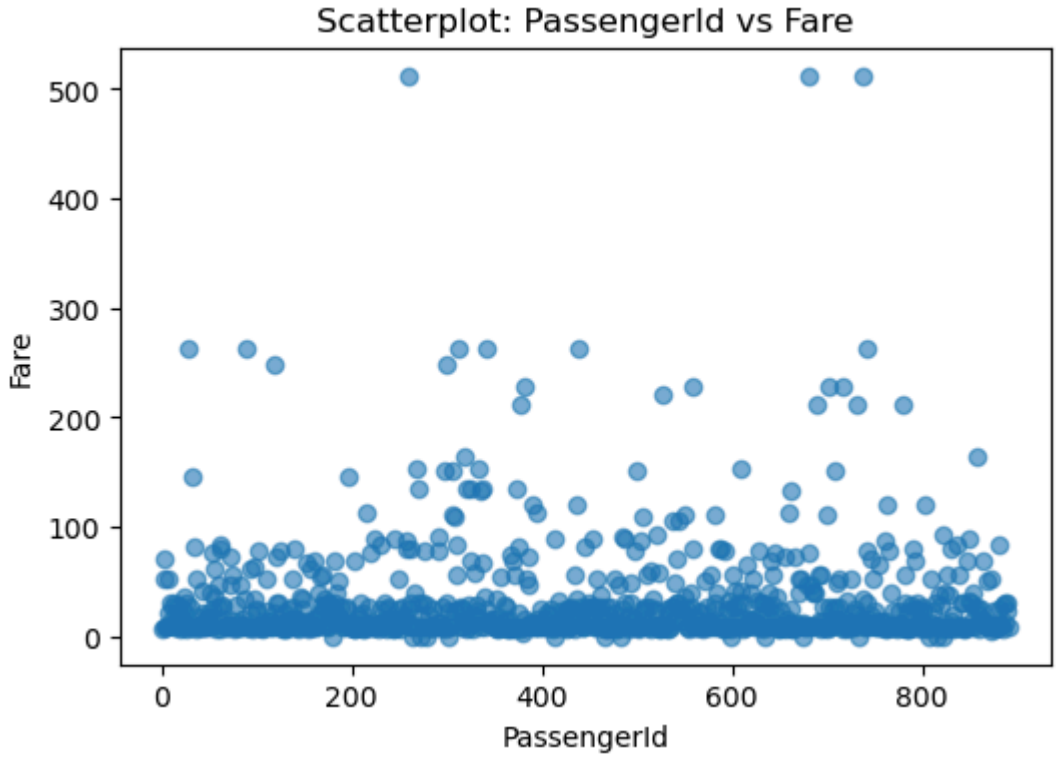

Scatterplot: PassengerId vs Survived
Survived
1
0
0
200
400
600
800
PassengerId

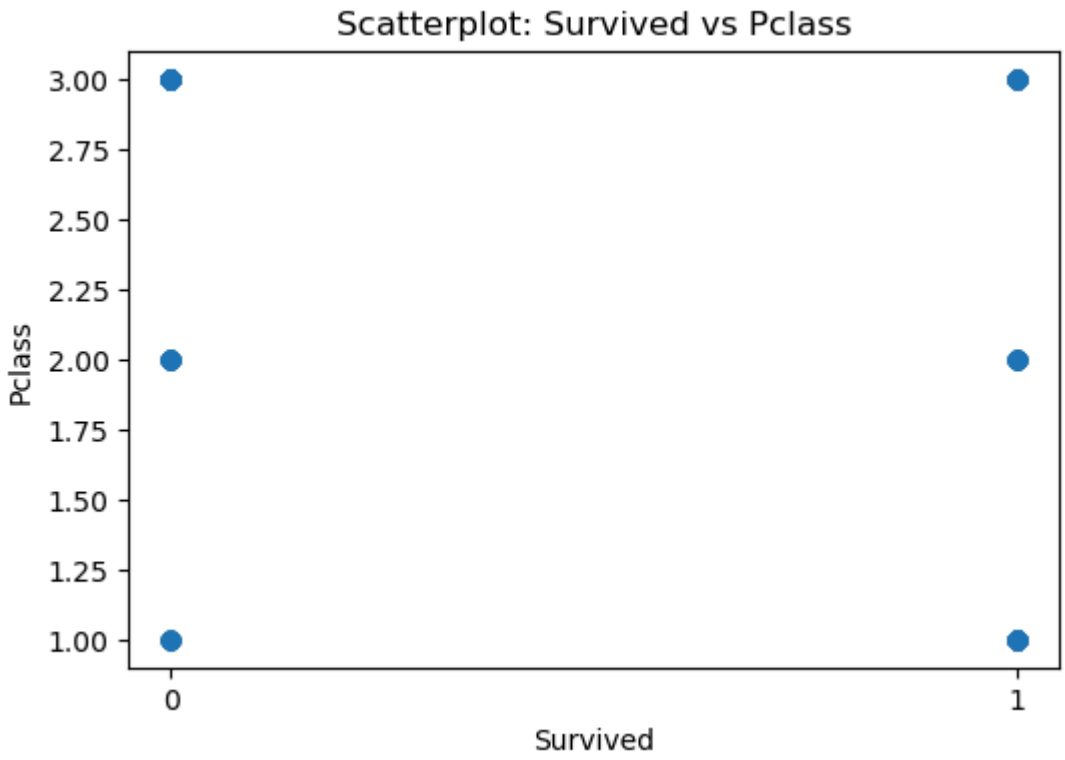

Scatterplot: PassengerId vs Pclass
Pclass
3.00
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00
0
200
400
600
800
PassengerId

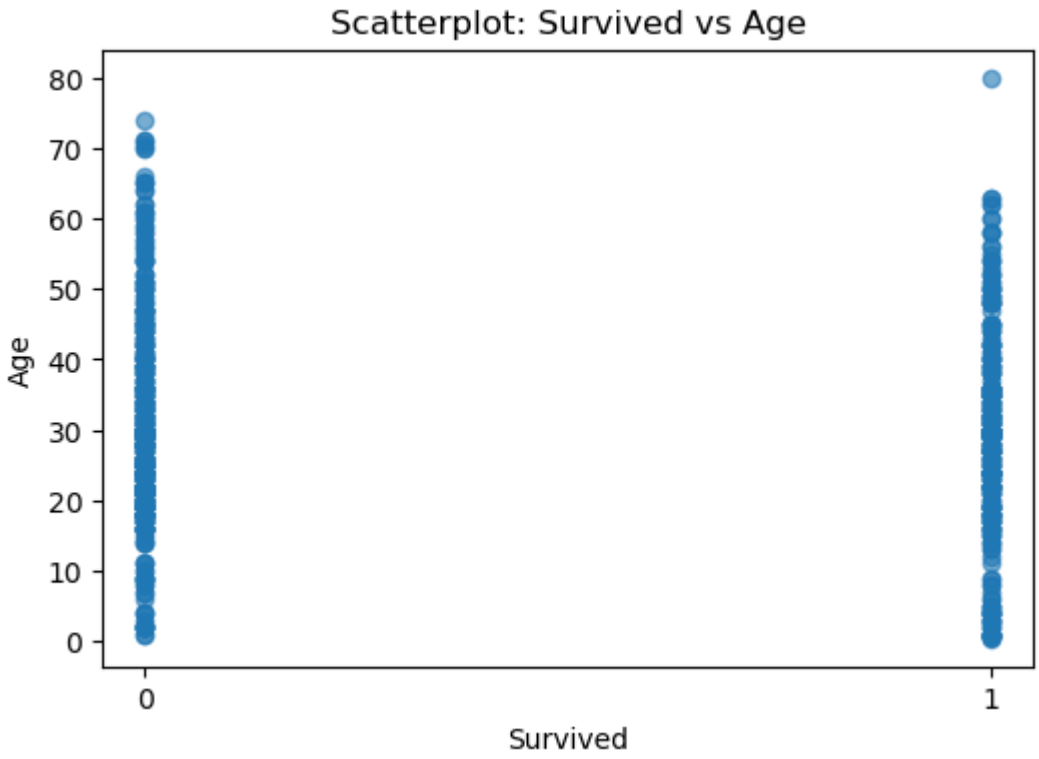

Scatterplot: PassengerId vs Age
Age
PassengerId
0
10
20
30
40
50
60
70
80
0
200
400
600
800

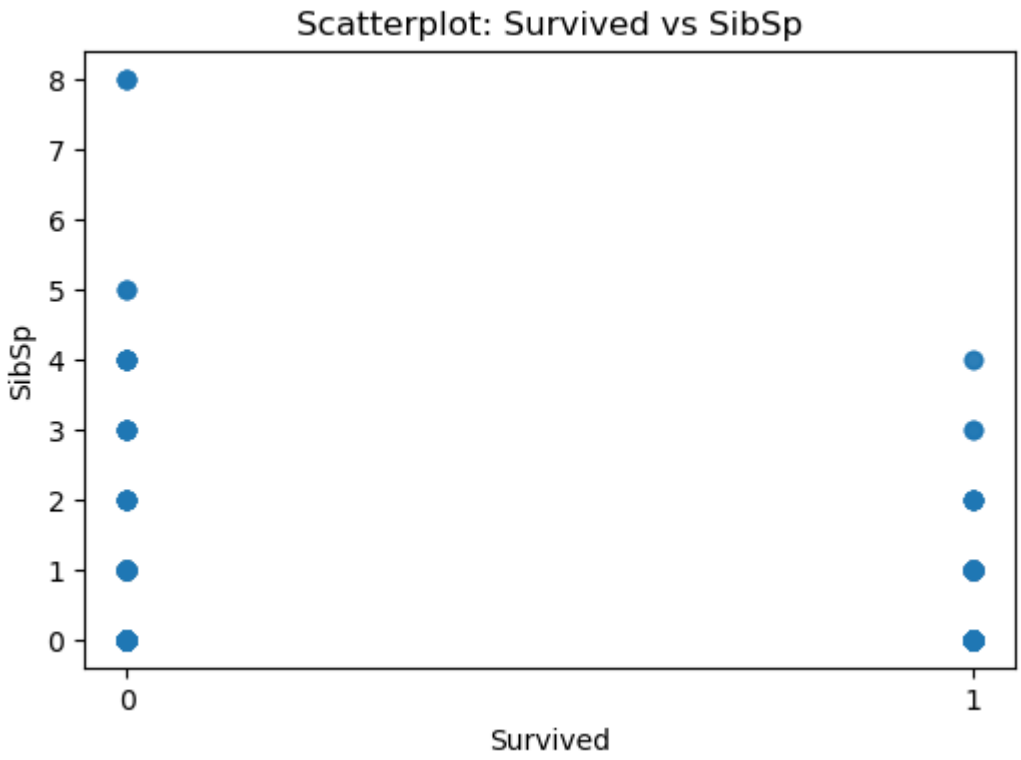

Scatterplot: PassengerId vs SibSp
SibSp
PassengerId
0
1
2
3
4
5
6
7
8
0
200
400
600
800

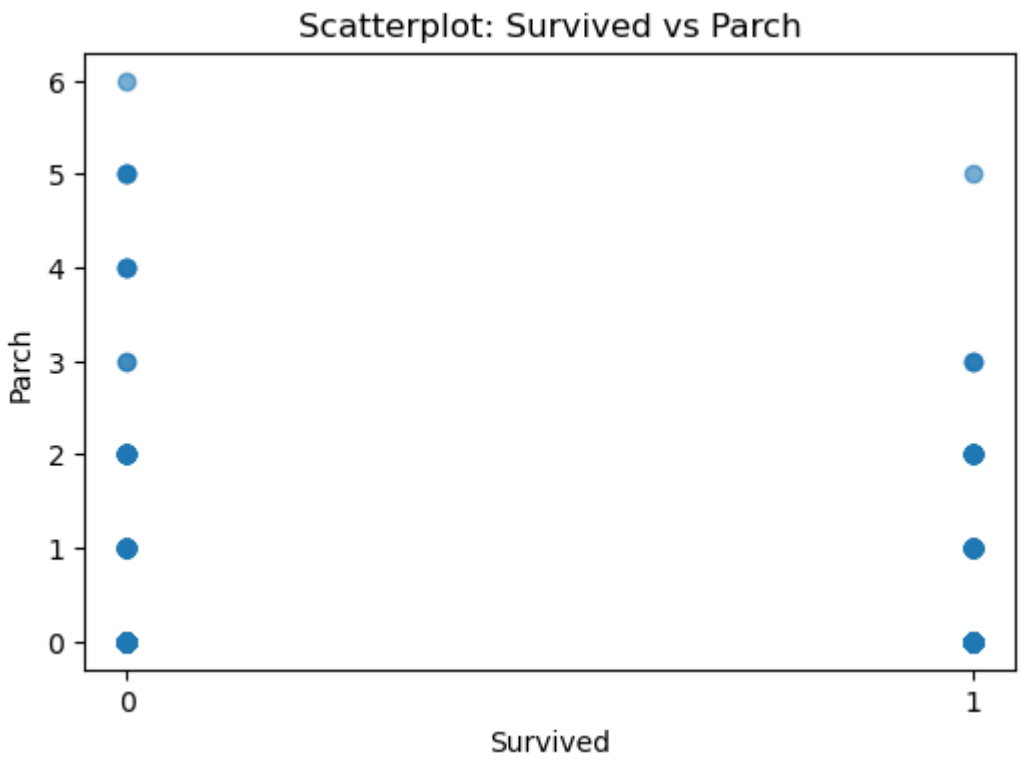

Scatterplot: PassengerId vs Parch
Parch
PassengerId
0
1
2
3
4
5
6
0
200
400
600
800

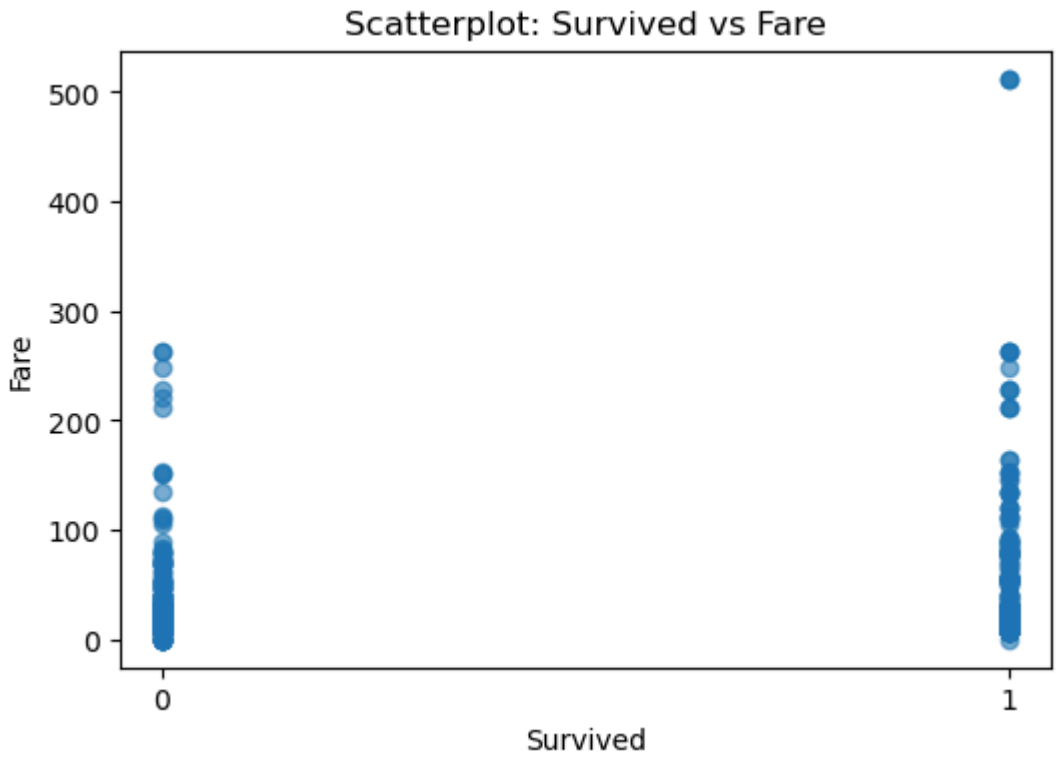

Scatterplot: PassengerId vs Fare
Fare
PassengerId
0
100
200
300
400
500
0
200
400
600
800

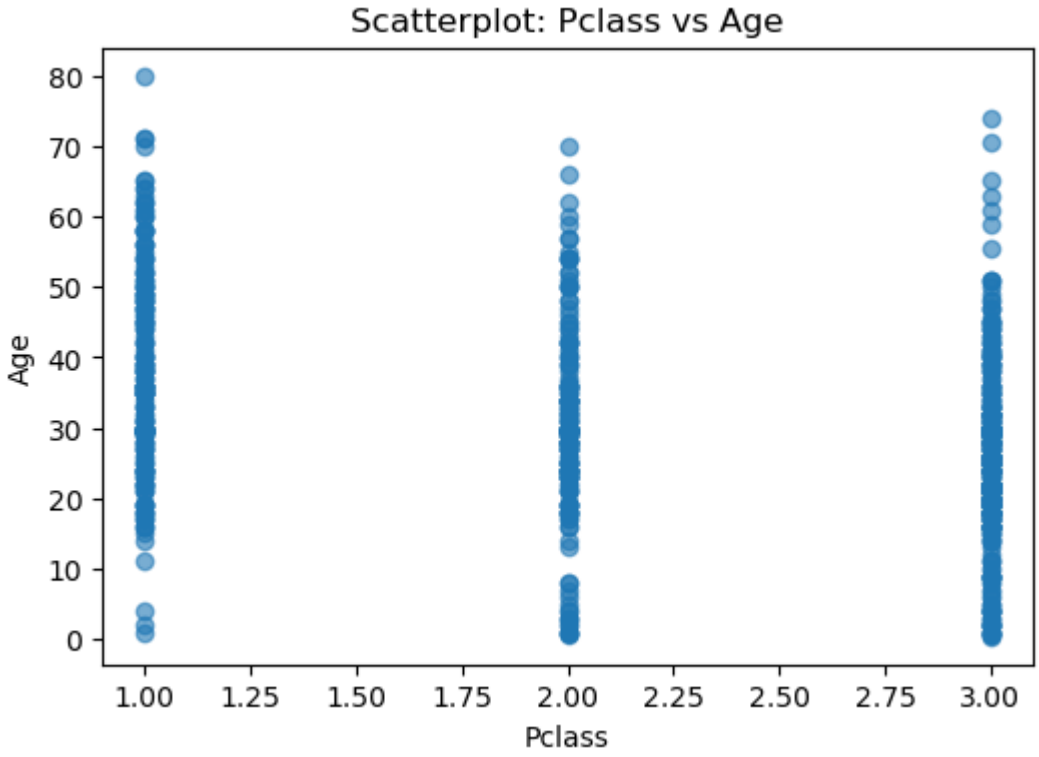

Scatterplot: Survived vs Pclass
3.00
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00
Pclass
0
1
Survived

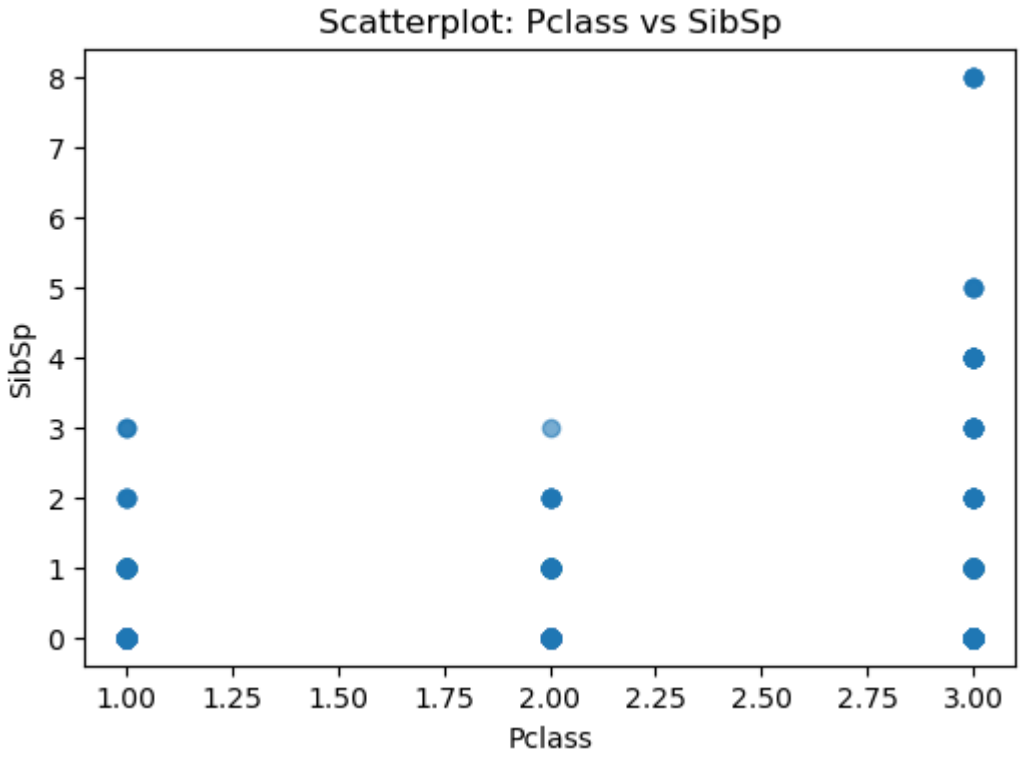

Scatterplot: Survived vs Age
80
70
60
50
40
30
20
10
0
Age
0
1
Survived

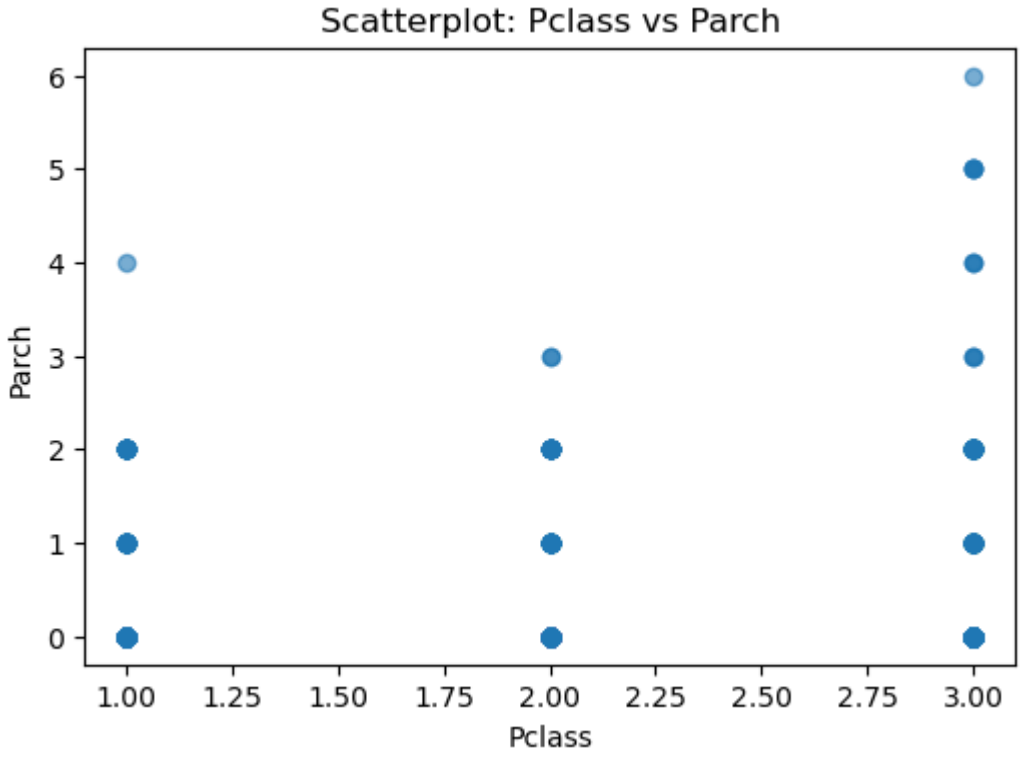

Scatterplot: Survived vs SibSp
SibSp
Survived
0
1
2
3
4
5
6
7
8

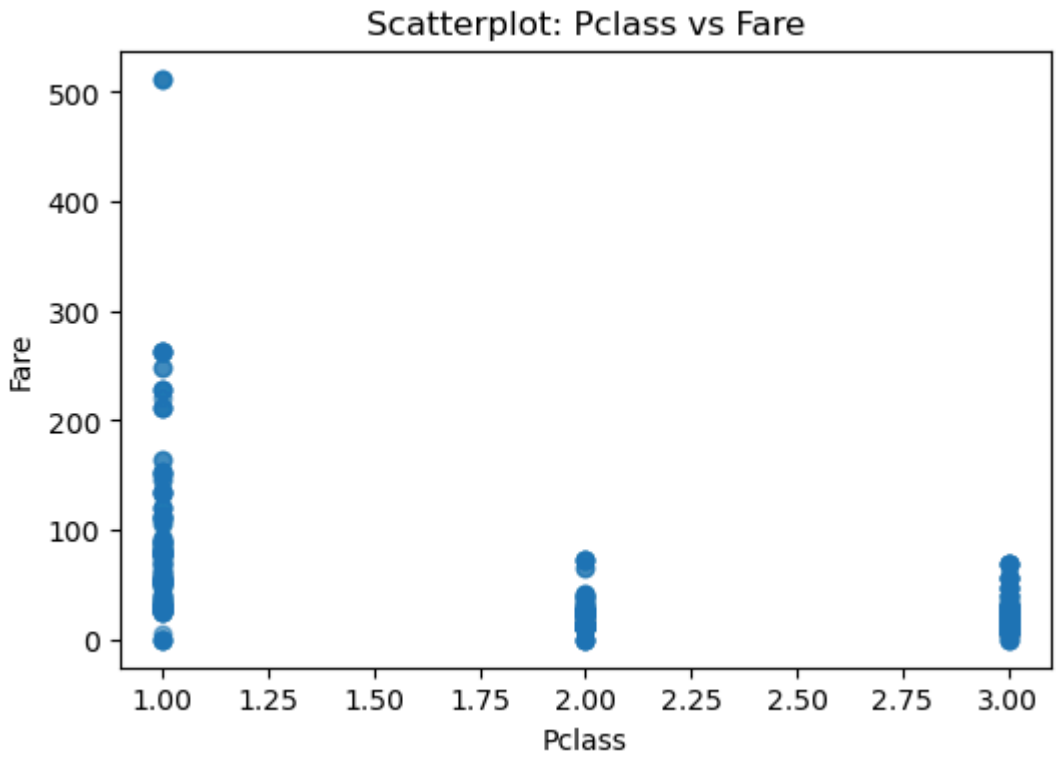

Scatterplot: Survived vs Parch
Parch
Survived
0
1
2
3
4
5
6

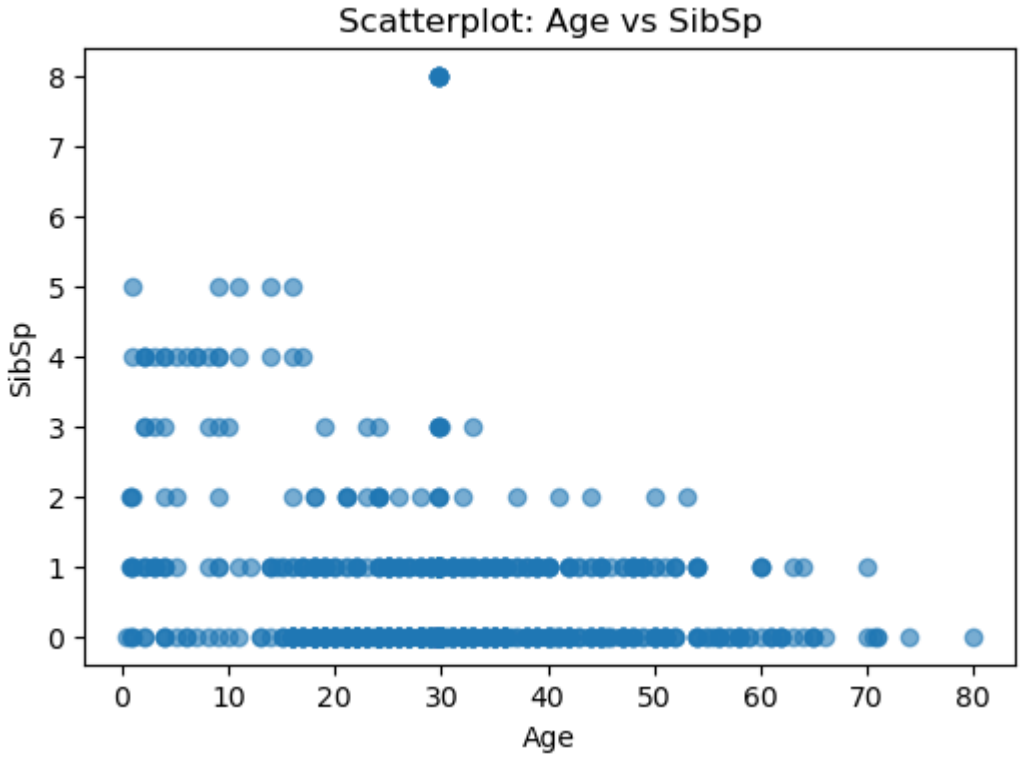

Scatterplot: Survived vs Fare
500
400
300
200
100
0
Fare
0
1
Survived

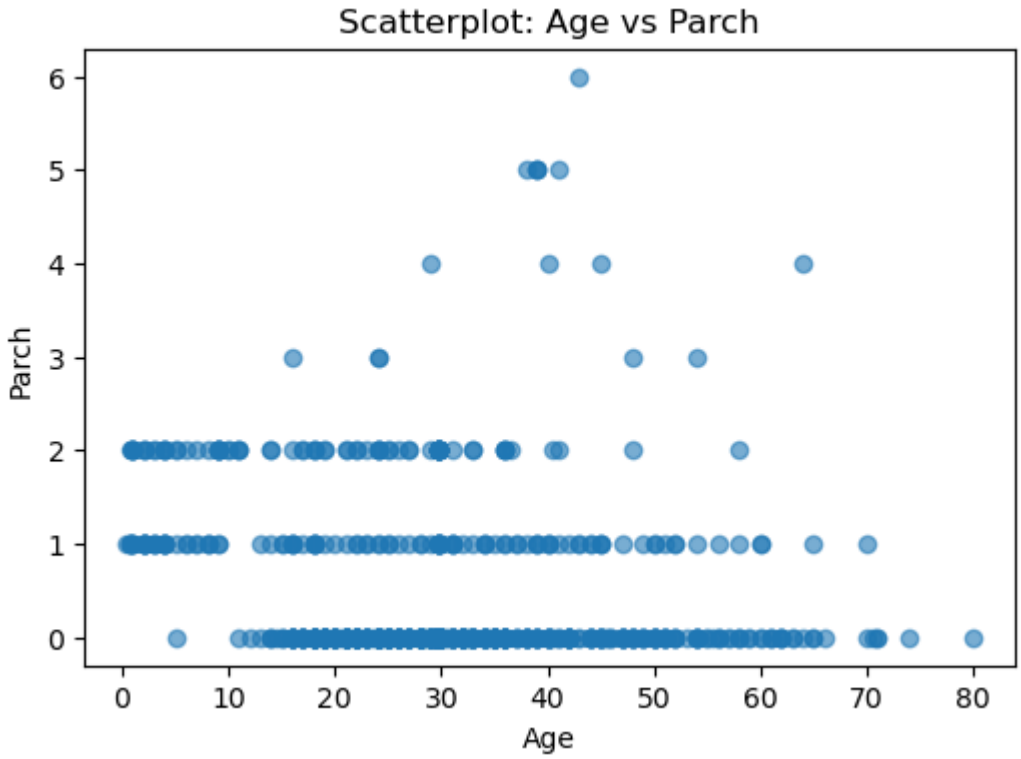

Scatterplot: Pclass vs Age
80
70
60
50
40
30
20
10
0
Age
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
Pclass

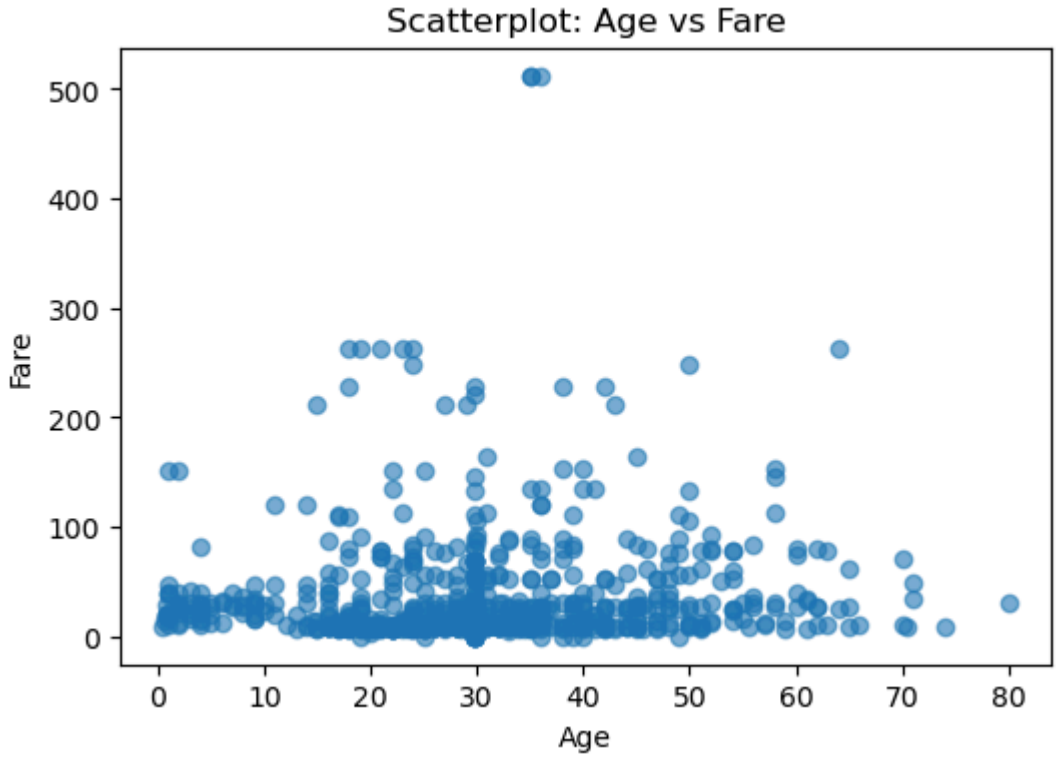

Scatterplot: Pclass vs SibSp
SibSp
Pclass
0
1
2
3
4
5
6
7
8
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00

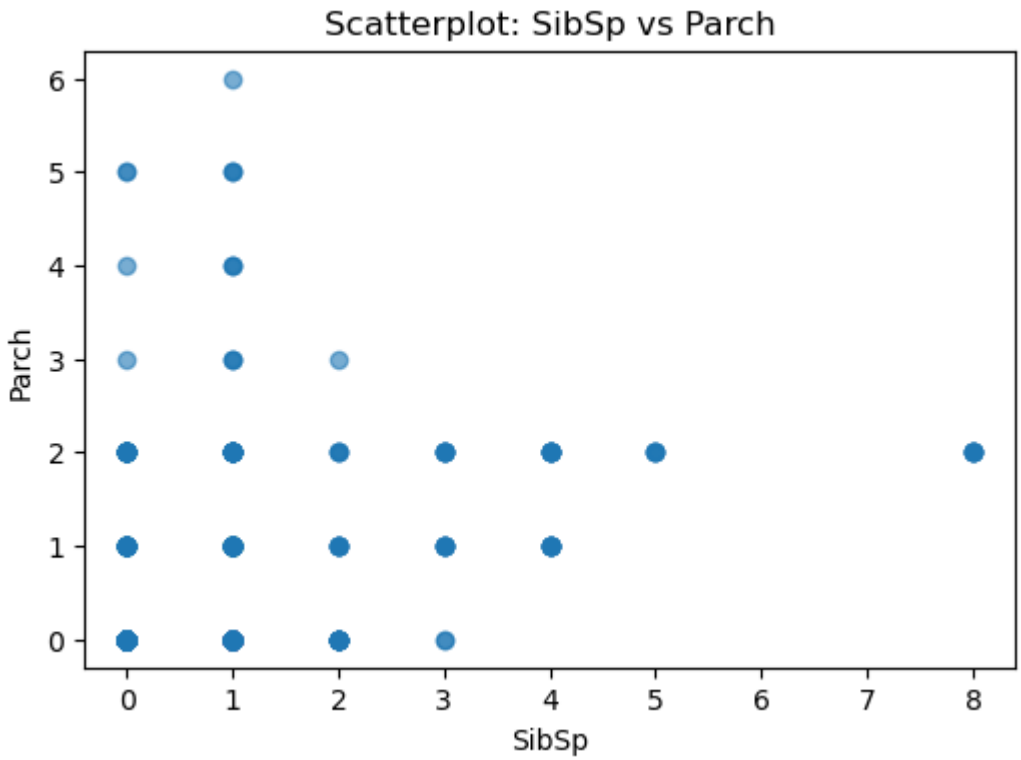

Scatterplot: Pclass vs Parch
Parch
Pclass
0
1
2
3
4
5
6
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00

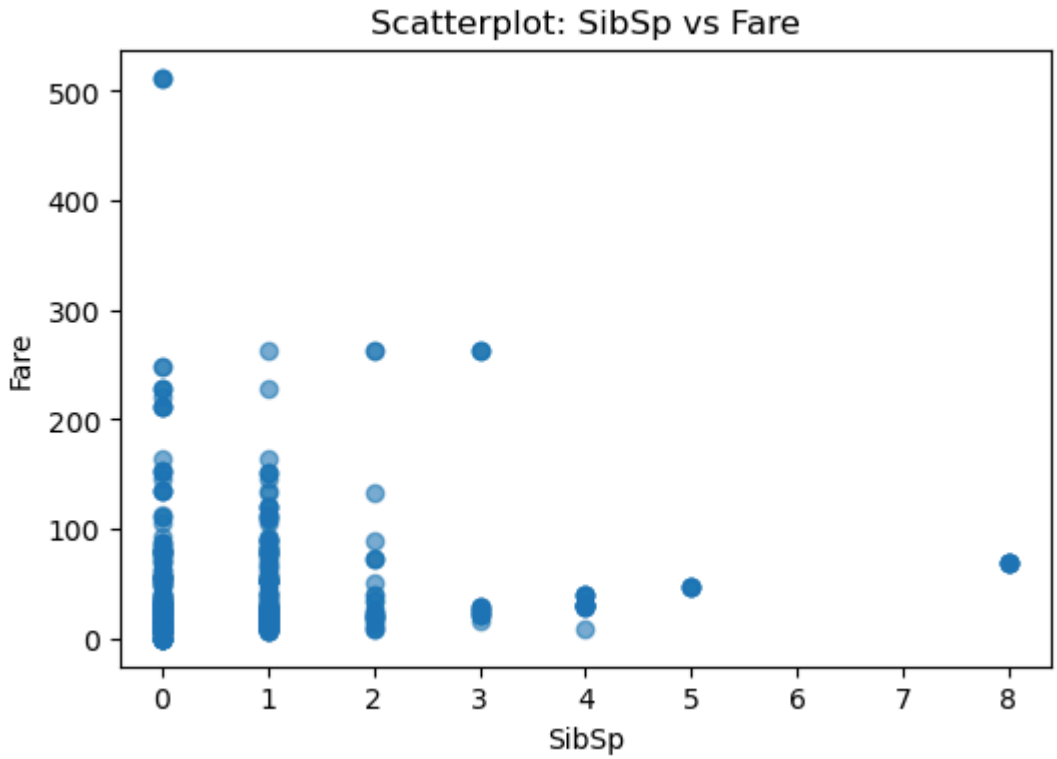

Scatterplot: Pclass vs Fare
Fare
Pclass
0
100
200
300
400
500
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00

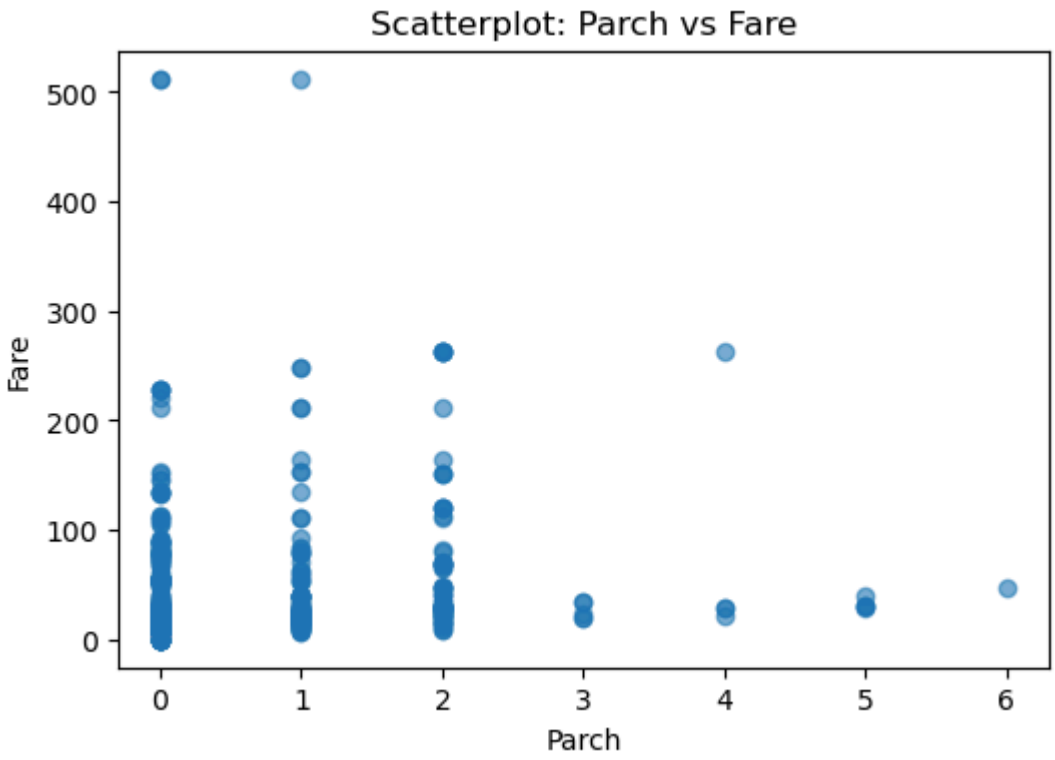

Scatterplot: Age vs SibSp
SibSp
Age
0
1
2
3
4
5
6
7
8
0
10
20
30
40
50
60
70
80


Scatterplot: Age vs Parch
Parch
Age
0
1
2
3
4
5
6
0
10
20
30
40
50
60
70
80


Scatterplot: Age vs Fare
Fare
Age
0
100
200
300
400
500
0
10
20
30
40
50
60
70
80


Scatterplot: SibSp vs Parch
Parch
SibSp
0
1
2
3
4
5
6
7
8


Scatterplot: SibSp vs Fare
Fare
SibSp
0
100
200
300
400
500
0
1
2
3
4
5
6
7
8


Scatterplot: Parch vs Fare
Fare
500
400
300
200
100
0
0
1
2
3
4
5
6
Parch

In [ ]: